

AI-Powered Transaction Processing Pipeline

System Design · Architecture · Scaling Strategy

RAKSHIT AGGARWAL: BTECH@IIITH,CSE

FastAPI · PostgreSQL · Redis · Celery · Gemini 2.5 Flash

What Does It Solve?



Ingest

Upload large
transaction CSVs

1



Clean

Remove duplicates,
normalize data

2



Detect

Flag anomalies &
unusual patterns

3



Classify

AI categorisation via
Gemini 2.5 Flash

4



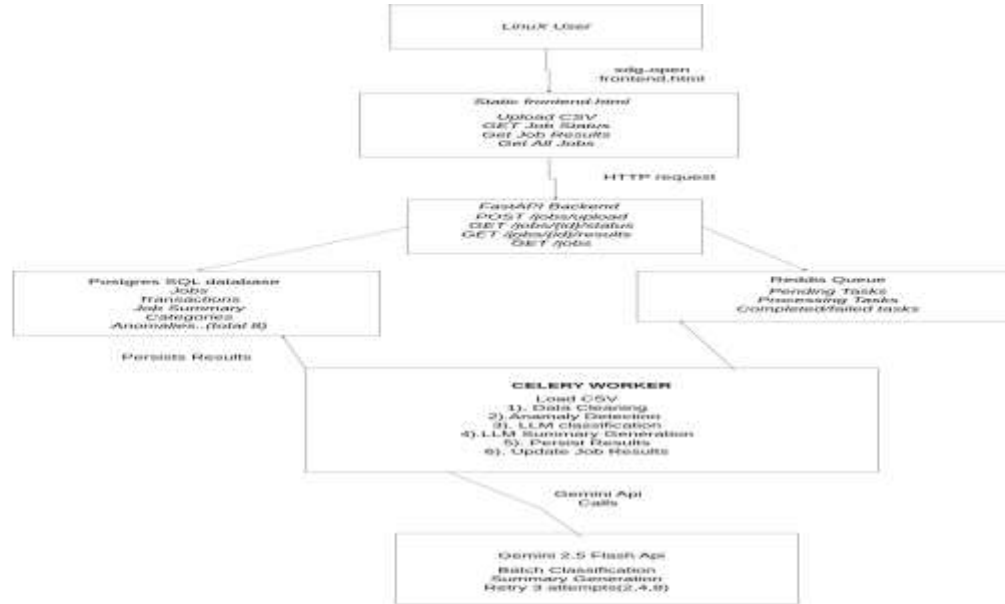
Summarise

Risk level +
spending narrative

5

All at scale · Reliably · With sub-100ms API response

System Architecture



Four Core Components

FastAPI (Port 8000)

- Validates 9-column CSV schema
- Creates Job record → returns ID instantly
- Sub-100ms response before processing starts

PostgreSQL

- 4 tables: Jobs, Transactions, Anomalies, Summaries
- ACID compliance for financial accuracy
- JSON for flexible anomaly metadata

Redis + Celery

- Decouples upload from heavy computation
- 4-stage chained pipeline per job
- Exponential backoff on task failure

Gemini 2.5 Flash

- Batched classification — 50+ txns per call
- 8 categories in one structured JSON response
- Retry ×3 · fallback to 'Other' on failure

6-Step Processing Pipeline



User receives Job ID immediately after upload — processing happens asynchronously in the background

API response: < 100ms

Total job time: 2–5 min

AI Layer — Gemini 2.5 Flash

Transaction Classification

Food

Shopping

Travel

Transport

Utilities

Cash Withdrawal

Entertainment

Other

Single batched call — all uncategorised txns sent together

AI Summary Generation

total_spend_inr / usd

top_merchants [×3]

anomaly_count

narrative (2–3 sentences)

risk_level low/medium/high

Retry ×3 · exponential backoff (2s / 4s / 8s) · fallback to local summary on failure

Database Schema

Jobs

id (UUID PK)

status (pending→completed)

filename, row_count

created_at, error_message

Transactions

job_id (FK)

txn_id, date, merchant, amount

category + llm_category

llm_failed (boolean)

Anomalies

job_id + transaction_id (FK)

anomaly_type, severity

details (JSONB)

created_at

JobSummaries

job_id (FK, UNIQUE)

total_spend_inr/usd

top_merchants (JSONB)

narrative, risk_level

Why This Architecture?



Async Processing

- Upload → Job ID in < 100ms
- Heavy work runs in background
- Users never wait for processing



Separation of Concerns

- FastAPI = HTTP layer only
- Celery = compute engine
- Each scales independently



Data Persistence

- Every txn saved to DB
- No loss on worker crash
- Full audit trail available



External API Resilience

- 3× retry with backoff(2,4,8 seconds)
- llm_failed flag avoids re-retry
- Local fallback if Gemini is down

Where It Breaks at 100× Scale

Current capacity: ~1,000 txns/hr → Target: 100,000 txns/hr

1

DB Connection Pool

Pool of 10 → exhausted at 1,000 concurrent

Fix: pgBouncer + pool to 50+

2

Single Celery Worker

1 job at a time → 50+ hr queue backlog

Fix: 10–20 worker instances + autoscale (scaling horizontally)

3

Gemini Rate Limit

1 req/s limit → 429 errors at scale

Fix: Larger batches + local BERT fallback

4

Redis Memory

Single node OOM → lost messages

Fix: Redis Cluster (3+3 nodes) + Sentinel

5

CSV Upload Size

Entire file loaded into RAM → OOM crash

Fix: Chunked streaming + S3 offload(Simple Storage service)

6

No Observability

Failures silent → no early warning

Fix: Prometheus + Grafana + ELK stack(provide metrics , dashboards for analysis and centralised logging)

Solutions at Scale(Re-Engineering Practices)

Solution	Key Change	Impact	Cost
Kubernetes Orchestration	Auto-scaling pods + load balancer (More fastapi and celery workers)	10–50× traffic spikes handled	+400%
DB Optimisation	Sharding + read replicas + pgBouncer(connection pool increase)	10–50× faster queries	+250%
Redis Cluster	3 master + 3 replicas + Sentinel	Zero OOM · 99.9% uptime	+150%
RabbitMQQueue	Guaranteed delivery + dead-letter queues	Zero message loss at scale	+\$200/ mo
Intelligent API Handling	Larger batches + circuit breaker + BERT fallback(lightweight model)	70% fewer API calls	+\$500/ mo
Observability Stack	Prometheus + Grafana + ELK	< 5 min incident response	+\$300/ mo

Key Takeaways

Built Right

- Async Celery pipeline → instant UX
- Modular services → independent scaling
- Structured LLM prompts → reliable JSON
- Fallbacks at every failure point

Scales Well

- Kubernetes + autoscaling pods
- Redis Cluster + RabbitMQ messaging
- DB sharding + read replicas
- 70% fewer LLM API calls via batching

Observed Clearly

- Prometheus metrics on every layer
- Grafana dashboards + mobile alerts
- ELK logs for incident forensics
- < 5 min MTTR at scale

Monolithic async → Fully distributed, auto-scaling microservices

LINK TO SUBMISSION:

Github Repo: [GithubRepo](#)

Drive(Draw.io): [Highlevel_image](#)